

Biophysics

Jakub Rydzewski

NCU Institute of Physics

Updated: May 28, 2026

Literature

- ▶ M. E. Tuckerman, *Statistical Mechanics: Theory and Simulation*, Oxford University Press (2016).
- ▶ D. Chandler, *Introduction to Modern Statistical Mechanics*, Oxford University Press (1987).

Monte Carlo Methods

Monte Carlo Methods

The main steps of the Monte Carlo sampling are:

- ▶ Choose randomly a new configuration in phase space based on a Markov chain.
- ▶ Accept or reject the new configuration, depending on the strategy used.
- ▶ Compute the physical quantity and add it to the averaging procedure.
- ▶ Repeat the previous steps until convergence.

Monte Carlo Methods

code/importance-sampling.py

$$I = \int_0^1 dx \int_0^{\sqrt{1-x^2}} dy = \frac{\pi}{4} \quad (1)$$

Monte Carlo Methods

- The integrals that must be evaluated in equilibrium statistical mechanics are generally of the form:

$$I = \int d\mathbf{x} \, \phi(\mathbf{x}) p(\mathbf{x}), \quad (2)$$

where $\mathbf{x}$ is an n -dimensional vector, $\phi(\mathbf{x})$ is an arbitrary function, and $p(\mathbf{x})$ is a function satisfying the properties of a probability distribution:

$$p(\mathbf{x}) \geq 0 \quad \text{and} \quad \int d\mathbf{x} \, p(\mathbf{x}) = 1. \quad (3)$$

- Eq. 2 represents the ensemble average of a physical observable in equilibrium statistical mechanics.

Monte Carlo Methods

- ▶ Let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M$ be a set of M n -dimensional vectors that are sampled from $p(\mathbf{x})$.
- ▶ The problem of sampling from $p(\mathbf{x})$ is a nontrivial one.
- ▶ For now, let us assume that such an algorithm exists. Then, an estimator:

$$\hat{I}_M = \frac{1}{M} \sum_{i=1}^M \phi(\mathbf{x}_i) \quad (4)$$

is such that:

$$\lim_{M \rightarrow \infty} \hat{I}_M = I. \quad (5)$$

- ▶ Eq. 5 is guaranteed by the *central limit theorem*.¹

¹For the derivation, see M. E. Tuckerman, *Statistical Mechanics: Theory and Simulation*, Oxford University Press (2016), p. 281.

Monte Carlo Methods

Central Limit Theorem

If $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_n$ are n random variables drawn from a population with overall mean μ and finite variance σ^2 , and if $\bar{\mathbf{X}}_n$ is the sample mean, the limiting form of the distribution, $\lim_{n \rightarrow \infty} \sqrt{n} \left(\frac{\bar{\mathbf{X}}_n - \mu}{\sigma} \right)$, is the standard normal distribution.

Monte Carlo Methods

- Example of sampling a simple distribution given by:

$$p(x) = ce^{-cx}, \quad (6)$$

on the interval $x \in [0, \infty)$.

- To sample from $p(x)$, we need $P(X)$ such that:

$$P(X) = \int_0^X dx \, ce^{-cx} = 1 - e^{-cX}, \quad (7)$$

and equate Eq. 7 to a random number $\xi \in [0, 1]$, then we can solve for X , which gives:

$$X = -\frac{1}{c} \log(1 - \xi). \quad (8)$$

- In general, we do not have such simple distribution to sample from in statistical mechanics.

Monte Carlo Methods: Examples

Estimator no 1

```
def pi_estimator(n_samples):  
    est = list()  
    n_inside = 0; n_all = 0  
    for i in range(n_samples):  
        x = np.random.uniform(-1.0, 1.0)  
        y = np.random.uniform(-1.0, 1.0)  
        dist = np.sqrt(np.power(x, 2) +  
                        np.power(y, 2))  
        if dist <= 1.0:  
            n_inside += 1  
        n_all += 1  
        est.append(4.0 * np.float(n_inside) /  
                  np.float(n_all))  
    return np.array(est)
```

Monte Carlo Methods: Examples

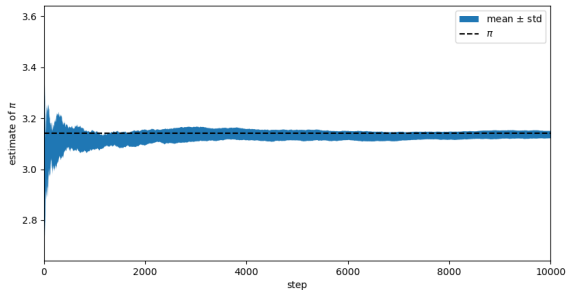


Figure 1: Estimator no 1.

Monte Carlo Methods: Examples

Estimator no 2

```
def pi_estimator_integral(n_samples):  
    est = list()  
    e = 0.0  
    for i in range(n_samples):  
        x = np.random.uniform(0.0, 1.0)  
        y = 4.0 * np.sqrt(1.0 - np.power(x, 2))  
        e += (y - e) / (np.float(i) + 1.0)  
        est.append(e)  
    return np.array(est)
```

Monte Carlo Methods: Examples

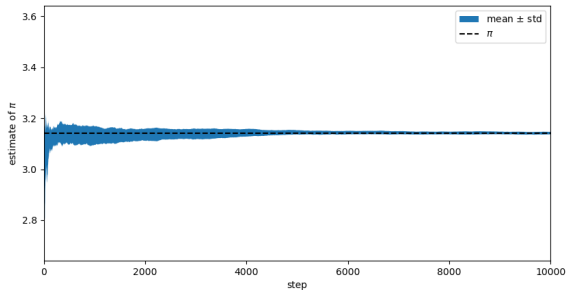


Figure 2: Estimator no 2.

Monte Carlo Methods: Examples

Estimator no 3

```
def pi_estimator_markov_chain(n_samples, step=0.1):
    est = list()
    n_inside = 0; n_all = 0;
    x = np.random.uniform(-1.0, 1.0)
    y = np.random.uniform(-1.0, 1.0)
    for i in range(n_samples):
        dx = np.random.uniform(-step, step)
        dy = np.random.uniform(-step, step)
        xn = x + dx; yn = y + dy
        if abs(xn) < 1.0 and abs(yn) < 1.0:
            x = xn; y = yn
        dist = np.sqrt(np.power(x, 2) + np.power(y, 2))
        if dist <= 1.0: n_inside += 1
    n_all += 1
    est.append(4.0 * np.float(n_inside) /
               np.float(n_all))
    return np.array(est)
```

Monte Carlo Methods: Examples

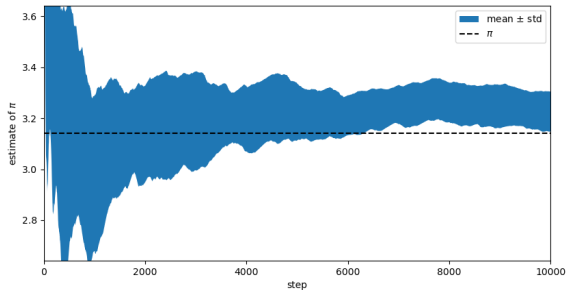


Figure 3: Estimator no 3.

$$M(RT)^2$$

Markov Chain

If the vectors $\mathbf{x}_1, \dots, \mathbf{x}_M$ are generated sequentially, and $\mathbf{x}_{i+1}$ is generated only based on the knowledge of $\mathbf{x}_i$. the sequence is called a Markov chain.

$M(RT)^2$

- ▶ For physical systems, a Markov chain must satisfy the condition of *detailed balance*, which ensures that the Markov process is microscopically reversible.
- ▶ $M(RT)^2$ is a rejection method.
- ▶ With the acceptance probability given by:

$$A(\mathbf{x}_i|\mathbf{x}_j) = \min [1, f(\mathbf{x}_i)/f(\mathbf{x}_j)] , \quad (9)$$

where $f(\mathbf{x})$ is the probability of the system being at $\mathbf{x}$.

- ▶ Sampling in the canonical ensemble:²

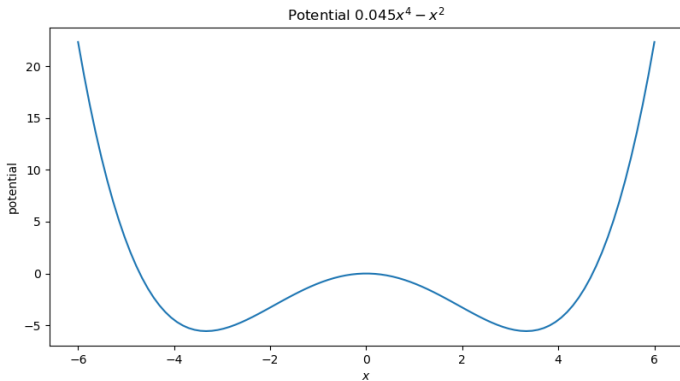
$$A(\mathbf{r}_i|\mathbf{r}_j) = \min \left[1, e^{-\beta[U(\mathbf{r}_i)-U(\mathbf{r}_j)]} \right] , \quad (10)$$

as $f(\mathbf{r}) \propto e^{-\beta U(\mathbf{r})}$.

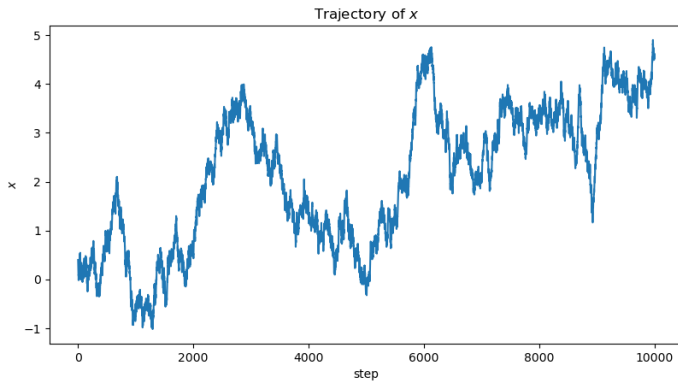
²Reminder: $\mathbf{r}$ are microscopic coordinates.

Particle on Double-Well Potential

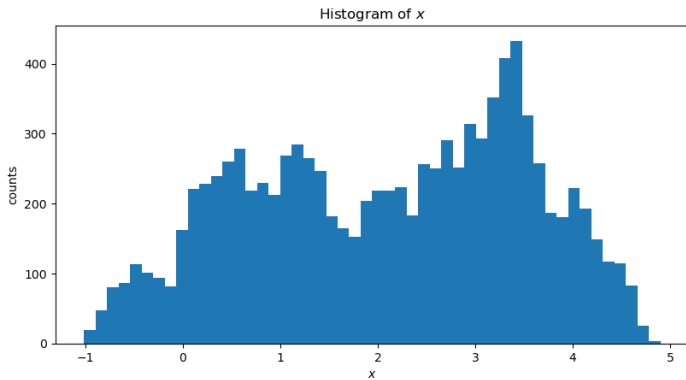
```
python mc-double-well.py --n_samples 10000 --temp 5
```



Particle on Double-Well Potential

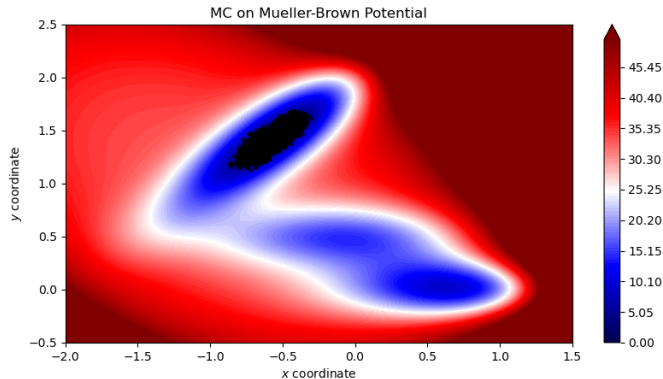


Particle on Double-Well Potential



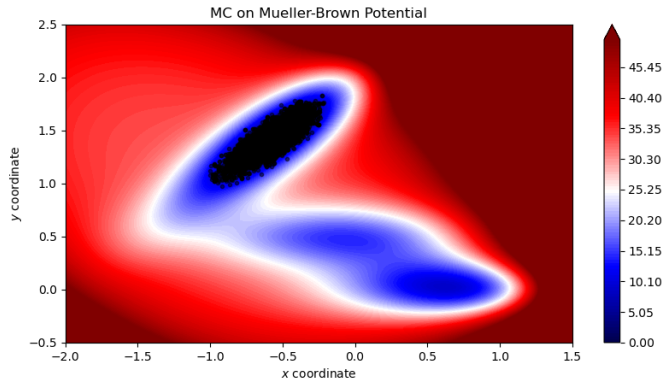
Particle on the Müller-Brown Potential

```
python mc-potential.py --n_samples 10000 --temp 1.0
```



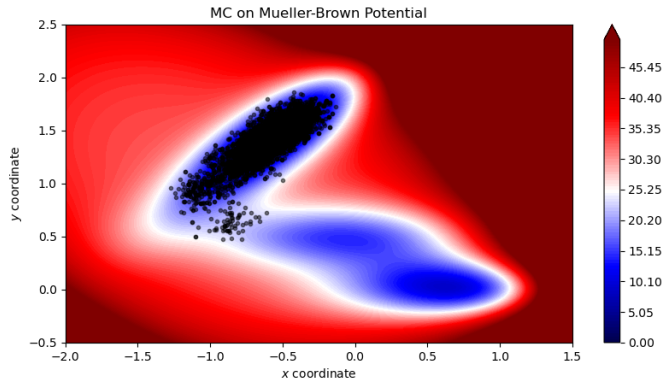
Particle on the Müller-Brown Potential

```
python mc-potential.py --n_samples 10000 --temp 2.0
```



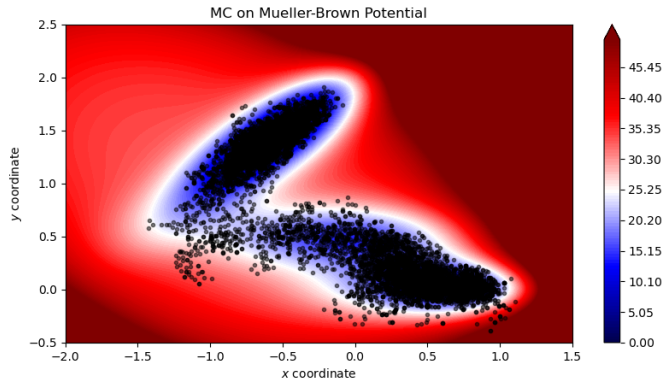
Particle on the Müller-Brown Potential

```
python mc-potential.py --n_samples 10000 --temp 3.0
```



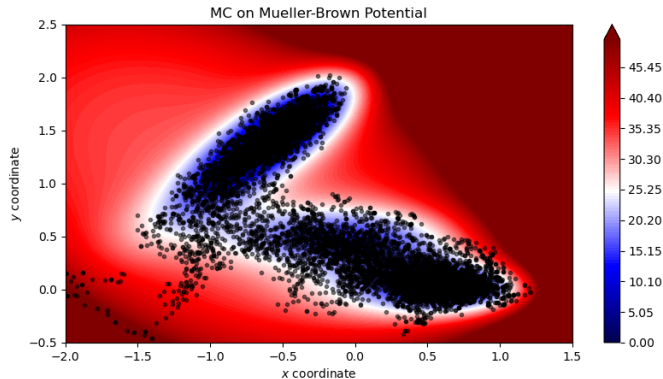
Particle on the Müller-Brown Potential

```
python mc-potential.py --n_samples 10000 --temp 4.0
```



Particle on the Müller-Brown Potential

```
python mc-potential.py --n_samples 10000 --temp 5.0
```



Molecular Dynamics

Molecular Dynamics

There are three main ingredients in molecular dynamics (MD):

- ▶ The algorithm to integrate the equations of motion,
- ▶ The model describing the interparticle interactions,
- ▶ The calculation of forces and energies from the model.

Verlet Algorithm

- The simplest approach to obtain a numerical integration scheme is to use a Taylor series:

$$\mathbf{r}_i(t + \Delta t) \approx \mathbf{r}_i(t) + \Delta t \mathbf{v}_i(t) + \frac{\Delta t^2}{2m_i} \mathbf{F}_i(t), \quad (11)$$

and

$$\mathbf{r}_i(t - \Delta t) \approx \mathbf{r}_i(t) - \Delta t \mathbf{v}_i(t) + \frac{\Delta t^2}{2m_i} \mathbf{F}_i(t). \quad (12)$$

- Adding Eq. 11 to Eq. 12, we get a velocity-independent Verlet algorithm:

$$\mathbf{r}_i(t + \Delta t) = 2\mathbf{r}_i(t) - \mathbf{r}_i(t - \Delta t) + \frac{\Delta t^2}{m_i} \mathbf{F}_i(t). \quad (13)$$

Velocity Verlet Algorithm

- Consider a shift in time in comparison to Eq. 11:

$$\mathbf{r}_i(t) \approx \mathbf{r}_i(t + \Delta t) - \Delta t \mathbf{v}_i(t + \Delta t) + \frac{\Delta t^2}{2m_i} \mathbf{F}_i(t + \Delta t), \quad (14)$$

which after some rearrangements takes the following form:

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \frac{\Delta t}{2m_i} [\mathbf{F}_i(t) + \mathbf{F}_i(t + \Delta t)]. \quad (15)$$

- This scheme allows to use both evolution of the positions and velocities.

Initial Conditions

- ▶ Initial coordinates.
- ▶ Initial velocities are sampled from a Maxwell-Boltzmann distribution, taking care to ensure that the sampled velocities are consistent with any constraints imposed on the system:

$$f(v) = \frac{m^{1/2}}{2\pi k_B T} e^{-mv^2/2k_B T}, \quad (16)$$

where v is the velocity of a particle of mass m at temperature T .

- ▶ An example of a Gaussian probability distribution.

Potential Energy Model

- Commonly used model of potential energy in MD for biological macromolecules is the following:

$$U(\mathbf{r}) = \sum_{\text{bond}} \frac{1}{2} K_{\text{bond}} (r - r_0)^2 \quad (17)$$

$$+ \sum_{\text{angle}} \frac{1}{2} K_{\text{bends}} (\theta - \theta_0)^2 \quad (18)$$

$$+ \sum_{\text{dihedral}} \sum_{n=0}^6 A_n [1 + \cos(C_n \phi + \delta_n)] \quad (19)$$

$$+ \sum_{i,j \in \text{non-bonded}} \left(4\epsilon_{ij} \left[\frac{\sigma_{ij}^{12}}{r_{ij}} - \frac{\sigma_{ij}^6}{r_{ij}} \right] + \frac{q_i q_j}{r_{ij}} \right). \quad (20)$$

- Force is given by $\mathbf{F} = -\partial U / \partial \mathbf{r}$.
- Called *force field* in MD.

Potential Energy Model

- Force field provides constraints into the systems.

`fig/ff.png`

General Scheme in MD



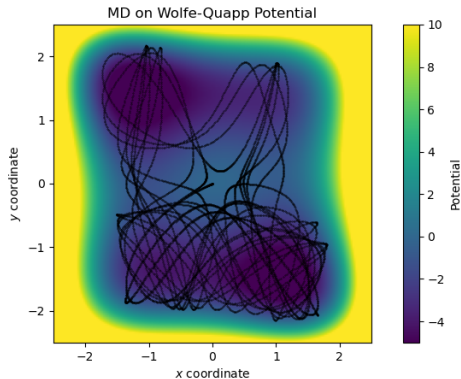
`fig/md-scheme.png`

Particle on the Wolfe-Quapp Potential

```
for i in range(0, args.n_samples-1):  
    position[i+1] = position[i]  
                  + velocity[i]*dt  
                  + 0.5*(force/m)*dt**2  
    p, f = U.eval(position[i+1])  
    velocity[i+1] = velocity[i]  
                  + 0.5/m*dt*(force+f)  
    p_energy[i+1] = p  
    force = f
```

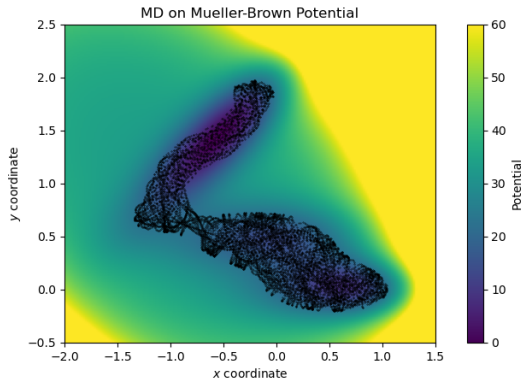
Particle on the Wolfe-Quapp Potential

```
python MD-WolfeQuapp.py --n_samples 10000 --dt 0.01 --mass 1
```

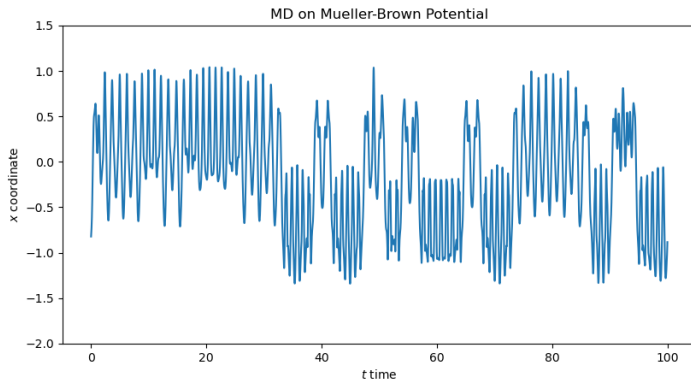


Particle on the Müller-Brown Potential

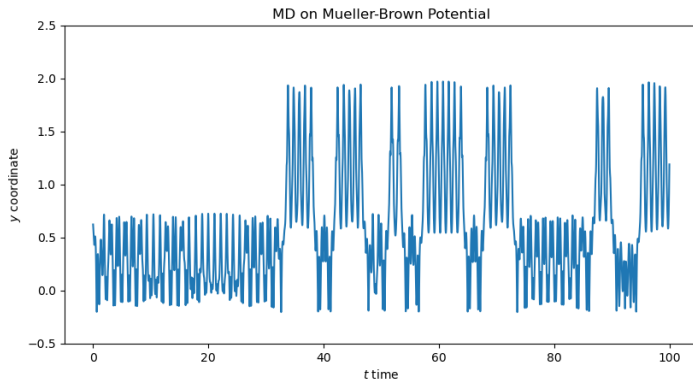
```
python MD-MuellerBrown.py --n_samples 10000 --dt 0.01 --mass 1
```



Particle on the Müller-Brown Potential



Particle on the Müller-Brown Potential



Alanine Dipeptide

- ▶ Benchmark system for MD methods.



fig/alanine.png

Figure 5: Alanine dipeptide with the dihedral angles.

Alanine Dipeptide

- An example of the .gro file with alanine dipeptide.

```
22
1ACE    HH31      1    1.075    1.111    1.585
1ACE      CH3      2    1.129    1.065    1.502
1ACE    HH32      3    1.070    0.991    1.448
1ACE    HH33      4    1.215    1.011    1.540
1ACE      C       5    1.162    1.167    1.395
1ACE      O       6    1.140    1.286    1.416
2ALA      N       7    1.230    1.118    1.289
2ALA      H       8    1.248    1.019    1.293
2ALA     CA       9    1.260    1.194    1.165
2ALA     HA      10    1.304    1.125    1.093
2ALA     CB      11    1.126    1.224    1.093
2ALA     HB1     12    1.078    1.312    1.135
2ALA     HB2     13    1.147    1.247    0.989
2ALA     HB3     14    1.073    1.129    1.094
2ALA      C      15    1.357    1.317    1.171
2ALA      O      16    1.392    1.372    1.068
3NME      N      17    1.404    1.353    1.294
3NME      H      18    1.350    1.322    1.373
3NME     CH3     19    1.517    1.445    1.315
3NME    HH31     20    1.495    1.544    1.278
3NME    HH32     21    1.546    1.457    1.420
3NME    HH33     22    1.606    1.405    1.267
```


Alanine Dipeptide

- Free energy can be calculated if the trajectory displays ergodicity as:

$$F(\Phi, \Psi) = -\beta^{-1} \log P(\Phi, \Psi), \quad (21)$$

where β is the inverse temperature and $P(\Phi, \Psi)$ is the probability of occurrence in a given state described by the dihedral angles.

Alanine Dipeptide

- ▶ Difference between the ergodic and non-ergodic trajectories.

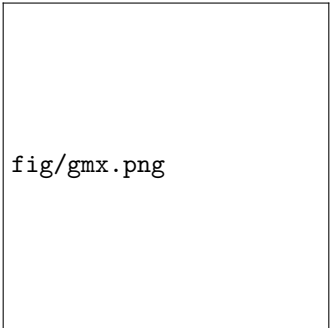
`fig/ala1-time.png`

Proteins in MD

- ▶ Proteins, DNA, viruses, membranes can be studied using MD.

fig/pes-protein.jpg

MD Engines



fig/gmx.png

- ▶ Gromacs <https://www.gromacs.org/>
- ▶ NAMD <https://www.ks.uiuc.edu/Research/namd/>
- ▶ OpenMM <https://openmm.org/>
- ▶ LAMMPS <https://lammps.sandia.gov/>

LAMMPS

*LAMMPS is a classical molecular dynamics (MD) code that models ensembles of particles in a liquid, solid, or gaseous state. It can model atomic, polymeric, biological, solid-state (metals, ceramics, oxides), granular, coarse-grained, or macroscopic systems using a variety of interatomic potentials (force fields) and boundary conditions. It can model 2d or 3d systems with only a few particles up to millions or billions.*³

³Source: LAMMPS Page: <https://lammmps.sandia.gov/>

LAMMPS: Executables for Linux

- ▶ Pre-built Ubuntu Linux executable (lammps-stable):

```
$ sudo add-apt-repository ppa:gladky-anton/lammps  
$ sudo apt-get update  
$ sudo apt-get install lammps-stable
```

- ▶ Pre-built OpenSuse Linux executable (lmp):

```
$ zypper install lammps
```

- ▶ Archlinux build-script (lmp):

```
$ git clone https://aur.archlinux.org/lammps.git  
$ cd lammps  
$ makepkg -s  
$ makepkg -i
```

- ▶ Also Conda:

```
$ conda config --add channels conda-forge  
$ conda create -n lammps-env  
$ conda activate lammps-env  
$ conda install lammps
```

LAMMPS: cmake

- ▶ However, it is suggested to build LAMMPS yourself.
- ▶ You can do this executing `./build-lammps.sh`.

```
#!/bin/bash
set -e

LAMMPS_VERSION=3Mar2020
Make_CPUs=4
URL=https://github.com/lammps/lammps/archive
wget ${URL}/stable_${LAMMPS_VERSION}.zip
unzip stable_${LAMMPS_VERSION}.zip
rm -f stable_${LAMMPS_VERSION}.zip
cd lammps-stable_${LAMMPS_VERSION}

mkdir build; cd build
cmake -DPKG_MANYBODY=yes \
      -DPKG_KSPACE=yes \
      -DPKG_MOLECULE=yes \
      -DPKG_RIGID=yes \
      ../cmake
make VERBOSE=1 -j ${Make_CPUs}
```

LAMMPS

- ▶ The tarball for this project can be downloaded here: [click](#).
- ▶ Contains the input file for the unbiased simulation.

```
irtg-school-mainz-2020/ $ tree
.
|-- data.nacl
|-- in.nacl
|-- nacl.psf
|-- run.sh
|-- start.lmp
|-- state.vmd
```

Source:

https://ves-code.github.io/doc-v2.7-irtg-school-2020/user-doc/html/ves_tutorials.html

LAMMPS

- ▶ The files comes from the TRR 146 IRTG School 2020 tutorial, and were prepared by Omar Valsson.
- ▶ We consider NaCl in aqueous solution.
- ▶ The dissociation barrier is expected to be around $2.5 k_B T$.
- ▶ Collective solvent motions play an important role in the transition.
- ▶ The system contains 1 Na, 1 Cl, and 106 water molecules (total 320 atoms).

LAMMPS

- The system:

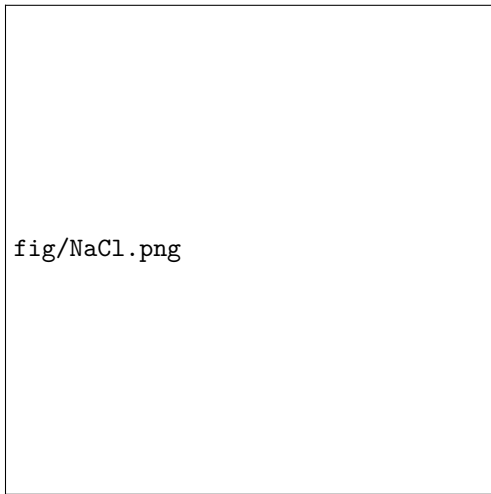


Figure 8: NaCl in water.

LAMMPS

- ▶ Download VMD ([click](#))
- ▶ Load the LAMMPS data file (`data.nacl`).
- ▶ Open VMD → Extensions → TK console.

```
$ topo readlammpsdata data.nacl  
$ play state.vmd
```

- ▶ We will run the simulation for 1 ns (see `start.lmp`)
- ▶ You can do this by executing `run.sh`.

```
#!/bin/bash
```

```
lmpexec=$HOME/lammps-stable_3Mar2020/build/lmp  
ncor=2
```

```
mpirun -np ${ncor} ${lmpexec} < start.lmp > out.lmp
```

`fig/plumed-logo.png`

PLUMED

PLUMED is an open-source, community-developed library that provides a wide range of different methods, which include:

- ▶ enhanced-sampling algorithms,
- ▶ free-energy methods,
- ▶ tools to analyze the vast amounts of data produced by molecular dynamics (MD) simulations.

These techniques can be used in combination with a large toolbox of collective variables that describe complex processes in physics, chemistry, material science, and biology.⁴

⁴Source: <https://www.plumed.org/>

PLUMED: Build for Linux

- Run `build-plumed.sh` (plumed):

```
#!/bin/bash
set -e

PLUMED_VERSION=2.6.2
Make_CPUs=2
URL=https://github.com/plumed/plumed2/archive
wget ${URL}/v${PLUMED_VERSION}.zip
unzip v${PLUMED_VERSION}.zip
cd plumed2-${PLUMED_VERSION}
INSTALL_DIR=${PWD}/install

./configure --prefix=${INSTALL_DIR}
make -j ${Make_CPUs}
make install
```

- Or use Conda:

```
$ conda config --add channels conda-forge
$ conda create -n lammps-env
$ conda activate lammps-env
$ conda install -c conda-forge plumed
```

PLUMED

- ▶ After you have finished the simulation, you should have `out.dcd` which stores the 1-ns trajectory of our system.
- ▶ Now we will use PLUMED to analyze our simulation.
- ▶ The distance between Na and Cl can be calculated using the following PLUMED action:

```
distance: DISTANCE ATOMS=319,320
```

- ▶ And the coordination number which represents the collective motion of the solvent wrt Na can be computed as:

```
COORDINATION ...  
GROUPA=319  
GROUPB=1-318:3  
SWITCH={RATIONAL R_0=0.315 D_MAX=0.5 NN=12 MM=24}  
NLIST  
NL_CUTOFF=0.55  
NL_STRIDE=10  
LABEL=coord  
... COORDINATION
```

- The coordination number calculates the following rational switching function:

$$f(x) = \frac{1 - \left(\frac{x}{r_0}\right)^n}{1 - \left(\frac{x}{r_0}\right)^m}, \quad (22)$$

where $r_0 = 0.315$, $n = 12$, and $m = 24$. You can see the function [here](#).

- Use this command to calculate the defined variables along the trajectory (stored in `run-driver.sh`):

```
plumed driver --plumed plumed-driver.dat \  
              --timestep 0.002 \  
              --trajectory-stride 100 \  
              --mf_dcd out.dcd
```


- You should obtain COLVAR-driver and see something like:

```
#! FIELDS time distance coord  
0.200000 0.568067 5.506808  
0.400000 0.500148 4.994588  
0.600000 0.449778 4.931140  
0.800000 0.528272 5.105816  
1.000000 0.474371 5.089863  
1.200000 0.430620 5.091551  
1.400000 0.470374 4.993886  
1.600000 0.458768 4.940097  
1.800000 0.471886 4.952868  
2.000000 0.489058 4.897593  
.  
.
```

LAMMPS

Exercises

1. Load the trajectory: in terminal `vmd out.dcd -psf nacl.psf`, then in VMD `pbw wrap -compound res -all`.
2. Change the visualization of atoms and render the system.
3. How the distance between Na and Cl changes in time?
4. Plot distances as a histogram (normalized). What distance value is the most probable?
5. From the histogram calculate $F = -\frac{1}{\beta} \log N$, where N is the histogram.
6. Where the barrier is located?
7. Perform the same analysis (from step 2) for the coordination number.

Enhanced Sampling

Enhanced Sampling: Source

- ▶ O. Valsson, P. Tiwary, and M. Parrinello, *Enhancing Important Fluctuations: Rare Events and Metadynamics from a Conceptual Viewpoint*, Annual Review of Physical Chemistry, 67:159-184, 2016.

Long Timescales

- ▶ Modeling the long-timescale behavior of complex dynamical systems is a fundamental task in physical sciences.
- ▶ MD simulations allow us to probe the spatiotemporal details of molecular processes, but the so-called sampling problem severely limits their usefulness in practice.
- ▶ This sampling problem comes from the fact that a typical free energy landscape is characterized by many metastable states separated by free energy barriers that are much higher than the thermal energy $k_{\text{B}}T$.

$$k_{\text{B}}T$$

- ▶ $k_{\text{B}}T$ is used in physics as a scale factor for energy values in molecular-scale systems.
- ▶ Inverse of $k_{\text{B}}T$ is called the inverse temperature and generally as $\beta = \frac{1}{k_{\text{B}}T}$.
- ▶ For a system in equilibrium in canonical ensemble, the probability of the system being in state with energy U is proportional to $e^{-\beta U}$.
- ▶ $k_{\text{B}}T$ is the amount of heat required to increase the thermodynamic entropy of a system by k_{B} .

$$k_{\text{B}}T$$

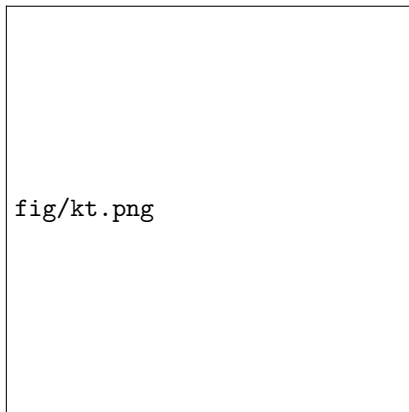


Figure 9: $k_{\text{B}}T$ in different units.⁵

⁵Source: [https://en.wikipedia.org/wiki/KT_\(energy\)](https://en.wikipedia.org/wiki/KT_(energy))

Metastable States

`fig/metastate.jpg`

Theory of Enhanced Sampling

- ▶ Consider a molecular system, described by microscopic coordinates $\mathbf{R}$ and a potential energy function $U(\mathbf{R})$.
- ▶ We limit our theory to the canonical ensemble (NVT).
- ▶ At equilibrium, the microscopic coordinates follow the Boltzmann distribution:

$$P(\mathbf{R}) = e^{-\beta U(\mathbf{R})} / \int d\mathbf{R} e^{-\beta U(\mathbf{R})}, \quad (23)$$

where β is the inverse of the thermal energy.

- ▶ We identify a small set of coarse-grained order parameters called *collective variables* (CVs):

$$\mathbf{s}(\mathbf{R}) = [s_1(\mathbf{R}), s_2(\mathbf{R}), \dots, s_d(\mathbf{R})]. \quad (24)$$

Collective Variables

- ▶ Also called *order parameters* or *coarse-grained variables* in some publications.
- ▶ Should be of lower dimension than microscopic coordinates $\mathbf{R}$.
- ▶ Can be any function of microscopic coordinates $s(\mathbf{R})$.
- ▶ For instance: angles, distances, contact map, dihedral angles, index along a predefined reaction coordinate, position, and many others.
- ▶ Can be periodic.
- ▶ Any differentiable function (to calculate forces) that discriminates between metastable states.
- ▶ Usually selecting CVs is a very difficult task.
- ▶ Nowadays machine learning can be used to find CVs: dimensionality reduction methods based on neural networks.

Theory of Enhanced Sampling

- The equilibrium marginal distribution of CVs is given by integrating out all other degrees of freedom:

$$P(\mathbf{s}) = \int d\mathbf{R} \delta[\mathbf{s} - \mathbf{s}(\mathbf{R})] P(\mathbf{R}), \quad (25)$$

where we use the sifting property of the Dirac δ -function.

- Eq. 25 is equivalent to:

$$P(\mathbf{s}) = \langle \delta[\mathbf{s} - \mathbf{s}(\mathbf{R})] \rangle, \quad (26)$$

where $\langle \cdot \rangle$ denotes an ensemble average.

- Up to an unimportant constant, the *free energy* surface (FES) is given by:

$$F(\mathbf{s}) = -\beta^{-1} \log P(\mathbf{s}). \quad (27)$$

Theory of Enhanced Sampling

- ▶ FES contains all the relevant information about the system, such as the location and relative stability of the metastable states and the free energy barriers separating them.
- ▶ The relative free energy difference between two metastable states A and B can be determined from $F(\mathbf{s})$ by integrating the probabilities of the two states:

$$\Delta F_{A,B} = -\frac{1}{\beta} \log \frac{\int_A d\mathbf{s} e^{-\beta F(\mathbf{s})}}{\int_B d\mathbf{s} e^{-\beta F(\mathbf{s})}}, \quad (28)$$

where the integrations are performed over the two regions in CV space that define the two metastable states A and B , respectively.

Theory of Enhanced Sampling

- For an ergodic system, the FES can equivalently be computed as:

$$F(\mathbf{s}) = -\frac{1}{\beta} \lim_{t \rightarrow \infty} N(\mathbf{s}, t), \quad (29)$$

where the normalized histogram is given as:

$$N(\mathbf{s}, t) = \frac{\int_0^t dt' \delta[\mathbf{s} - \mathbf{s}(\mathbf{R}(t'))]}{\int_0^t dt'}. \quad (30)$$

Theory of Enhanced Sampling

- ▶ CV-based enhanced sampling methods overcome the sampling problem by introducing an external bias potential $V(\mathbf{s}(\mathbf{R}))$ acting in CV space.
- ▶ This leads to sampling according to a *biased* distribution:

$$P_V(\mathbf{R}) = \frac{e^{-\beta[U(\mathbf{R})+V(\mathbf{s}(\mathbf{R}))]}}{\int d\mathbf{R} e^{-\beta[U(\mathbf{R})+V(\mathbf{s}(\mathbf{R}))]}} \quad (31)$$

- ▶ Compare it to Eq. 23.
- ▶ Idea of Torrie and Valleau, 1977.

Theory of Enhanced Sampling

- At convergence, CVs follow a biased distribution:

$$\begin{aligned} P_V(\mathbf{s}) &= \int d\mathbf{R} \, \delta[\mathbf{s} - \mathbf{s}(\mathbf{R})] P_V(\mathbf{R}) \\ &= \frac{e^{-\beta[F(\mathbf{s})+V(\mathbf{s})]}}{\int d\mathbf{s} \, e^{-\beta[F(\mathbf{s})+V(\mathbf{s})]}} , \end{aligned} \quad (32)$$

which is easier to sample.

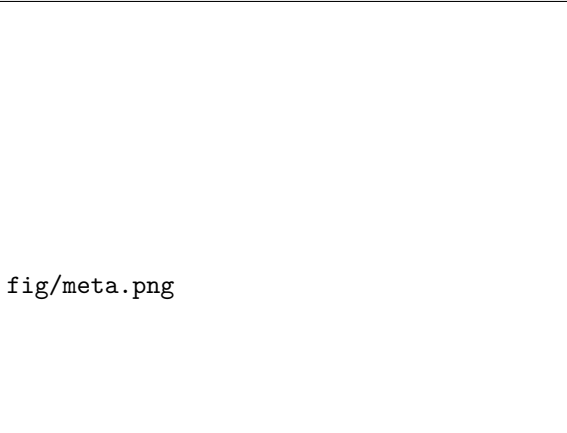
- The standard reweighting from importance sampling applies here:

$$P(\mathbf{R}) \propto P_V(\mathbf{R})w(\mathbf{s}(\mathbf{R})) = P_V(\mathbf{R})e^{\beta V(\mathbf{s}(\mathbf{R}))}. \quad (33)$$

- CV-based methods differ in how they construct the bias potential and which kind of biased CV sampling they obtain at convergence.

Metadynamics

- ▶ Introduced by Laio and Parrinello in 2002.⁶
- ▶ A dynamical scheme in which energy basins are filled in using a time-dependent potential that depends on the history of the system's trajectory.
- ▶ Once a basin is filled in, the system is driven into the next basin.



fig/meta.png

Metadynamics

- Let's see how the unbiased probability distribution looks:

$$P(\mathbf{s}) = \left\langle \prod_{\alpha=1}^n \delta[f_{\alpha}(\mathbf{r}) - \mathbf{s}_{\alpha}] \right\rangle. \quad (34)$$

- We can replace the phase space average with a time average over a trajectory:

$$P(\mathbf{s}) = \lim_{\tau \rightarrow \infty} \frac{1}{\tau} \int_0^{\tau} dt \prod_{\alpha=1}^n \delta[f_{\alpha}(\mathbf{r}(t)) - \mathbf{s}_{\alpha}], \quad (35)$$

under the assumption that the dynamics is ergodic.

Metadynamics

- In metadynamics, we express the Dirac function as the limit of a Gaussian function:

$$\delta[x - a] = \lim_{\sigma \rightarrow 0} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-a)^2/2\sigma^2}. \quad (36)$$

- Then the probability can be written as:

$$P(\mathbf{s}) = \lim_{\tau \rightarrow \infty} \lim_{s \rightarrow 0} \frac{1}{\sqrt{2\pi\Delta s^2\tau}} \cdot \int_0^\tau dt \prod_{\alpha=1}^n \exp \left[-\frac{(\mathbf{s}_\alpha - f_\alpha(\mathbf{r}(t)))^2}{2\Delta s^2} \right]. \quad (37)$$

Metadynamics

- Numerically, Eq. 37 is written as a discrete sum:

$$P(\mathbf{s}) \approx \frac{1}{\sqrt{2\pi\Delta s^2\tau}} \cdot \sum_{k=0}^{N-1} \exp \left[- \sum_{\alpha=1}^n \frac{(\mathbf{s}_\alpha - f_\alpha(\mathbf{r}(t)))^2}{2\Delta s^2} \right]. \quad (38)$$

- Eq. 38 suggests a form of the bias potential.
- Consider a bias potential of the form:

$$V(\mathbf{r}, t) = w \sum_{t=k\tau} \exp \left[- \sum_{\alpha=1}^n \frac{(f_\alpha(\mathbf{r}) - f_\alpha(\mathbf{r}_\tau(t)))^2}{2\Delta s^2} \right], \quad (39)$$

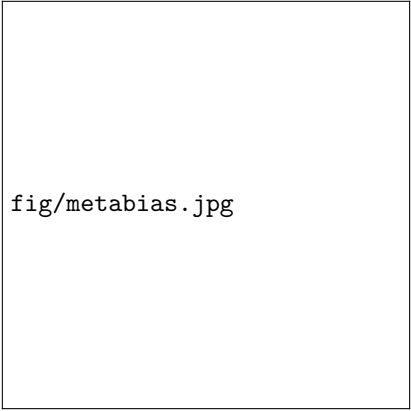
where $k = 1, 2, \dots$ and τ is the time stride of depositing the bias.

- $\mathbf{r}_\tau(t)$ is the biased time evolution of the complete microscopic coordinates up to time t .

Metadynamics

- In the simplest variant of metadynamics, the negative of Eq. 39 is taken as an approximation to the free energy:

$$F(\mathbf{q}) = -\lim_{t \rightarrow \infty} V(\mathbf{q}(\mathbf{r}), t). \quad (40)$$



fig/metabias.jpg

Metadynamics

- Mapping a high-dimensional PES to a low-dimensional FES.

(a) PES (C. Dellago)

(b) FES (O. Valsson)

Well-Tempered Metadynamics

- We can build a bias potential using the following expression:

$$V(\mathbf{s}) = - \left(1 - \frac{1}{\gamma}\right) F(\mathbf{s}), \quad (41)$$

where $\gamma \geq 1$ is a parameter called bias factor.

- Then we can insert Eq. 41 to the formula for the biased probability distribution:

$$\begin{aligned} P_V(\mathbf{s}) &\propto \exp[-\beta(F(\mathbf{s}) + V(\mathbf{s}))] \\ &= \exp[-\beta(F(\mathbf{s}) - (1 - 1/\gamma)F(\mathbf{s}))] \\ &= \exp\left[-\frac{\beta}{\gamma}F(\mathbf{s})\right] = [P(\mathbf{s})]^{1/\gamma}, \end{aligned} \quad (42)$$

which is the so-called well-tempered distribution.

Well-Tempered Metadynamics

- ▶ Fluctuations enhanced.
- ▶ Easier to cross barriers.
- ▶ Flatten the sampling comparing to the unbiased distribution $P(\mathbf{s})$.

Well-Tempered Metadynamics

- Using Eq. 42 and:

$$F_V(\mathbf{s}) = -\frac{1}{\beta} \log P_V(\mathbf{s}), \quad (43)$$

we can obtain an expression for the free energy under the bias potential:

$$F_V(\mathbf{s}) = \frac{1}{\gamma} F(\mathbf{s}), \quad (44)$$

where we ignore unimportant additive constants.

- We sample on an effective FES where the barriers are reduced by a factor of γ .
- Optimal γ is such that effective barriers are around $k_B T$.

Well-Tempered Metadynamics

- ▶ Taking the limit $\gamma \rightarrow \infty$ gives:

$$V(\mathbf{s}) = -F(\mathbf{s}) \quad (45)$$

and

$$P_V(\mathbf{s}) \propto 1, \quad (46)$$

which gives uniform sampling of CVs.

- ▶ Completely reduces of free energy barriers.
- ▶ Not optimal: Spend a lot of time sampling irrelevant regions in free energy.
- ▶ Better to enhance CVs with a finite value of γ .

Well-Tempered Metadynamics

- To achieve such sampling, we can iteratively build a bias by periodically updating it:

$$V_n(\mathbf{s}) = V_{n-1}(\mathbf{s}) + G(\mathbf{s}, \mathbf{s}_n) \exp \left[-\frac{1}{\gamma - 1} \beta V_{n-1}(\mathbf{s}_n) \right], \quad (47)$$

where:

$$G(\mathbf{s}, \mathbf{s}_n) = W_0 \exp \left[-\sum_i^d \frac{(s_i - s_{n,i})^2}{2\sigma_i^2} \right] \quad (48)$$

is a Gaussian biasing kernel of height W_0 that is centered at the current CV location $\mathbf{s}_n$.

Well-Tempered Metadynamics

- The above update is performed every N_G steps.
- Between steps n and $n + 1$, the bias is:

$$V_n(\mathbf{s}) = \sum_{k=1}^n W_k \exp \left[- \sum_i \frac{(s_i - s_{k,i})^2}{2\sigma_i^2} \right], \quad (49)$$

where W_k is the Gaussian height for the Gaussian deposited at step k :

$$W_k = W_0 \exp \left[- \frac{1}{\gamma - 1} \beta V_{k-1}(\mathbf{s}_k) \right], \quad (50)$$

where the scaling factor decreases to zero as $1/n$:

$$\exp \left[- \frac{1}{\gamma - 1} \beta V_{n-1}(\mathbf{s}_n) \right] \rightarrow \frac{1}{n}. \quad (51)$$

Well-Tempered Metadynamics

- ▶ The bias reaches a quasi-stationary state, which is important for reweighting.
- ▶ It can be proven that in the long-time limit the bias potential is:

$$V(\mathbf{s}, t) = - \left(1 - \frac{1}{\gamma} \right) F(\mathbf{s}) + c(t), \quad (52)$$

where $c(t)$ is a time-dependent constant that can be estimated during a simulation.

- ▶ To perform reweighting, we need to account for time-dependence of the bias:

$$P(\mathbf{R}, t) = P_V(\mathbf{R}) e^{\beta[V(\mathbf{s}(\mathbf{R}), t) - c(t)]}. \quad (53)$$

Well-Tempered Metadynamics

- Any thermodynamic quantity O can be calculated as:

$$\langle O(\mathbf{R}) \rangle = \left\langle O(\mathbf{R}) e^{\beta[V(\mathbf{s}(\mathbf{R})) - c(t)]} \right\rangle_V, \quad (54)$$

where the time-dependent constant $c(t)$ is calculated as:

$$c(t) = \frac{1}{\beta} \log \frac{\int d\mathbf{s} \exp \left[\frac{\gamma}{\gamma-1} \beta V(\mathbf{s}, t) \right]}{\int d\mathbf{s} \exp \left[\frac{1}{\gamma-1} \beta V(\mathbf{s}, t) \right]}, \quad (55)$$

which is valid when the bias has reached a quasi-stationary state.

Well-Tempered Metadynamics

- It can be shown that a time-independent estimator for $F(\mathbf{s})$ is given by:

$$F(\mathbf{s}) = - \left(\frac{\gamma}{\gamma - 1} \right) V(\mathbf{s}, t) + \frac{1}{\beta} \log \int d\mathbf{s} \exp \left[\frac{\gamma}{\gamma - 1} \beta V(\mathbf{s}, t) \right] \quad (56)$$

- The nontempered limit $\gamma \rightarrow \infty$ corresponds to:

$$V(\mathbf{s}) = -F(\mathbf{s}), \quad (57)$$

which means that the bias fully compensates the underlying free energy landscape.

- This was the idea that guided the first version of metadynamics in which the strength of the biasing kernel was kept constant at each iteration.

Well-Tempered Metadynamics

- ▶ Alanine dipeptide in vacuum with backbone dihedral angles Φ and Ψ as CVs.
- ▶ Two metastable basins (or three), $C7_{\text{eq}}$ and $C7_{\text{ax}}$.
- ▶ Barrier around 34 kJ/mol ($16 k_{\text{B}}T$).
- ▶ Mean transition time around 28 μs .

Well-Tempered Metadynamics

- ▶ 300 times shorter trajectory.

Well-Tempered Metadynamics

Biasing Na-Cl

- ▶ Updated files for the enhanced sampling simulation are provided [here](#).
- ▶ We want to construct a bias potential $V(\mathbf{s})$ on the Na-Cl distance using WT-MetaD.
- ▶ We will run the simulation for 10 ns.
- ▶ The bias is given by:

$$V(\mathbf{s}) = - \left(1 - \frac{1}{\gamma} \right) F(\mathbf{s}), \quad (58)$$

so the system evolve under an effective free energy $F(\mathbf{s})/\gamma$.

Biasing Na-Cl

► Biasing the distance using PLUMED:

```
d1: DISTANCE ATOMS=319,320
metad: METAD ...
  ARG=d1
  SIGMA=0.02
  HEIGHT=1
  BIASFACTOR=5
  TEMP=300.0
  PACE=500
  GRID_MIN=0.2
  GRID_MAX=1.0
  GRID_BIN=300
  CALC_RCT
...
```

Biasing Na-Cl

- We also limit the exploration of the CV space by using an upper wall bias $V_w(\mathbf{s})$:

$$V_w(s) = \kappa(s - s_0)^2 \quad (59)$$

if $s > s_0$ and 0 otherwise.

```
d1:  DISTANCE  ATOMS=319,320
uwall:  UPPER_WALLS ...
      ARG=d1
      AT=0.6
      KAPPA=2000.0
      EXP=2
      EPS=1
      OFFSET=0.
...
```

Biasing Na-Cl

- ▶ Run the simulation using: `./run.sh` (as before).
- ▶ Estimate the free energy profile using HILLS file that resulted from the simulation:

```
plumed sum_hills --hills HILLS --min 0.1 --max 0.8 \  
                --bin 300 --stride 100
```

Biasing Na-Cl

- ▶ We should observe two states: associated and dissociated.
- ▶ Is the simulation converged?
- ▶ What energy should the system have while in the dissociated state?
- ▶ Why the free energy starts to rise for distance > 0.6 nm?

Reweighting Na-Cl simulation

- The weights to reweight the simulation are of the following form:

$$w \propto e^{\beta[V(\mathbf{s},t)-c(t)+V_w(\mathbf{s})]}, \quad (60)$$

where $V_w(\mathbf{s})$ is the wall bias appended to the simulation.

```
# Read COLVAR file
distance: READ FILE=COLVAR IGNORE_TIME VALUES=d1
metad: READ FILE=COLVAR IGNORE_TIME VALUES=metad.rbias
uwall: READ FILE=COLVAR IGNORE_TIME VALUES=uwall.bias

# Define weights
weights: REWEIGHT_BIAS TEMP=300 ARG=metad.rbias,uwall.bias
```

- A script to reweight the simulation is `plumed-driver.dat`.
- Execute it using: `plumed -driver --plumed
plumed-driver.dat --noatoms`

Biasing Na-Cl

- ▶ Plot the free energy from `histo.dat`.
- ▶ Compare it to the free energy generated without reweighting.
- ▶ Does the reweighting fix the free energy?